

OAuth – پروتکل احراز هویت و مجوزدهی

خلاصه و ترجمه فارسی نکات کلیدی

بر اساس: OAuth: Complete Technical Research – Fazel Mohammad Ali Pour (۲۰۲۱)

بخش ۱: احراز هویت در مقابل مجوزدهی

۱.۱ تعاریف پایه

این دو مفهوم اغلب با هم اشتباه گرفته می‌شوند، اما تفاوت اساسی دارند:

مثال	سوال پاسخ داده شده	تعریف	مفهوم
ورود با نام کاربری و رمز عبور؛ ارائه پاسپورت	آیا تو همانی هستی که ادعا می‌کنی؟	تأیید اینکه کاربر یا سیستم چه کسی است – اثبات هویت	احراز هویت (AuthN)
کاربر وارد شده می‌تواند پست بخواند اما پست دیگران را حذف نکند	چه چیزی مجاز به دسترسی هستی؟	تعیین اینکه موجودیت احراز هویت شده چه کاری مجاز است – اعطای دسترسی	مجوزدهی (AuthZ)

تشبیه ساده: احراز هویت مثل نشان دادن کارت شناسایی در ورودی است (اثبات هویت). مجوزدهی مثل دستبند رنگی است که بعد از آن می‌گیرید – تعیین می‌کند به کدام قسمت‌ها دسترسی دارید.

۱.۲ چرا این تمایز برای OAuth مهم است

- OAuth 2.0 صرفاً یک چارچوب مجوزدهی است؛ برای احراز هویت طراحی نشده
- OAuth 2.0: تفویض مجوزدهی – «اپ X ممکن است به فایل‌های Google Drive شما دسترسی داشته باشد»
- OpenID Connect: تفویض احراز هویت – «تأیید کنید این شخص همان کسی است که ادعا می‌کند»
- access_token اصلی OAuth نباید هرگز برای اثبات هویت استفاده شود
- id_token در OIDC یک JWT امضا شده است که مخصوص تأیید هویت طراحی شده

۱.۳ ضدالگوی رمزعبور (قبل از OAuth)

قبل از OAuth، برنامه‌های شخص ثالث مستقیماً نام کاربری و رمز عبور کاربران را می‌خواستند. این مشکلات را ایجاد می‌کرد:

- برنامه‌ها رمزهای خام ذخیره می‌کردند – نقض امنیتی یک برنامه، همه سرویس‌های مرتبط را در خطر می‌انداخت
 - بدون دسترسی دقیق – یا دسترسی کامل یا هیچ
 - بدون مکانیزم لغو – تغییر رمزعبور همه دسترسی‌های شخص ثالث را به‌طور همزمان لغو می‌کرد
 - فیشینگ – برنامه‌ها از درخواست‌های قانونی رمزعبور قابل تمایز نبودند
- OAuth این مشکلات را با معرفی مفهوم تفویض مجوزدهی از طریق توکن‌ها حل کرد.

بخش ۲: تاریخچه و تکامل OAuth

۲.۱ جدول زمانی تکامل

تغییر کلیدی	نقطه عطف	سال
اولین استاندارد برای تفویض مجوزدهی؛ امضاهای HMAC-SHA1	OAuth 1.0 Draft	۲۰۰۷
رفع آسیب‌پذیری Session Fixation؛ معرفی oauth_verifier	OAuth 1.0a (RFC 5849)	۲۰۰۹
طراحی کامل مجدد؛ bearer tokens؛ انواع مختلف grant؛ ساده‌تر اما نیازمند HTTPS	OAuth 2.0 (RFC 6749)	۲۰۱۲
لایه هویتی روی OAuth 2.0 برای احراز هویت استاندارد	OpenID Connect 1.0	۲۰۱۴
محافظت از Authorization Code flow برای کلاینت‌های عمومی	PKCE (RFC 7636)	۲۰۱۵
جریان برای دستگاه‌های TV/IoT/CLI	Device Authorization Grant (RFC 8628)	۲۰۱۹
بهترین شیوه‌های امنیتی فعلی؛ PKCE اجباری	OAuth 2.0 Security BCP (RFC 9700)	۲۰۲۵/۲۰۲۰
تثبیت نسخه ۲/۰ و حذف جریان‌های منسوخ؛ PKCE همه‌جا	OAuth 2.1 (Draft)	+۲۰۲۱
اثبات مالکیت توکن – جلوگیری از سرقت bearer token	DPoP (RFC 9449)	۲۰۲۳

۲.۲ انواع Grant در OAuth 2.0

توضیح	کاربرد	نوع Grant
امن‌ترین روش – سرور کد را دریافت و با secret آن را با توکن تعویض می‌کند	وب‌اپ‌های server-side	Authorization Code
مانند Authorization Code اما بدون client secret – با چالش رمزنگاری جایگزین می‌شود	اپ‌های موبایل و SPA	Authorization Code + PKCE
مستقیم بین دو سرور بدون دخالت کاربر	Machine-to-Machine (M2M)	Client Credentials
دستگاه کد نمایش می‌دهد، کاربر در مرورگر تأیید می‌کند، دستگاه polling می‌کند	CLI، IoT، Smart TV	Device Code Flow
توکن مستقیم در URL برمی‌گردد – منسوخ شده؛ ناامن	منسوخ (Deprecated)	Implicit Grant
اپ مستقیم نام کاربری و رمز عبور می‌گیرد – هدف OAuth را نقض می‌کند	منسوخ (Deprecated)	Resource Owner Password

۲.۳ انواع توکن

محل نگهداری	هدف	طول عمر	توکن
بلافاصله استفاده می‌شود	تبادل برای دریافت توکن‌ها	بسیار کوتاه (ثانیه تا دقیقه، یک‌بار مصرف)	Authorization Code
حافظه یا storage امن؛ هرگز نه cookie	اعتبارنامه Bearer برای API	کوتاه (۶۰-۵ دقیقه)	Access Token
HttpOnly cookie رمزگذاری شده	دریافت access token جدید بدون ورود مجدد	بلند (روز تا ماه)	Refresh Token
پارس و اعتبارسنجی کنید - خام ذخیره نکنید	JWT امضا شده تأییدکننده هویت کاربر (OIDC)	کوتاه (مانند access token)	ID Token (OIDC)

بخش ۳: آسیب‌پذیری‌ها و بردارهای حمله

۳.۱ CVE ها و آسیب‌پذیری‌های مهم

راه‌حل	توضیح	نام / CVE
OAuth 1.0a: معرفی oauth_verifier	مهاجم یک Request Token از پیش ایجاد می‌کند سپس کاربر را فریب می‌دهد تا آن را تأیید کند	Session Fixation (۲۰۰۹)
state پارامتر CSRF؛ اعتبارسنجی دقیق redirect	جریان OAuth فیسبوک اجازه دزدی توکن cross-domain را می‌داد	CVE-2018-1000531 (Facebook)
اعتبارسنجی iss، aud، exp، nonce؛ استفاده از کتابخانه OIDC	اعتبارسنجی نامناسب id_token اجازه تصاحب حساب را می‌داد	CVE-2019-1020020 (Auth0)
مقایسه دقیق URI شامل scheme، path، host	دور زدن اعتبارسنجی redirect_uri اجازه open redirect و دزدی توکن را داد	CVE-2021-27580 (SAP)
اعتبارسنجی scope‌های اعطاشده با درخواست شده	آسیب‌پذیری scope escalation اجازه دسترسی غیرمجاز به scope‌های بالاتر را داد	CVE-2022-3213 (GitLab)
احراز هویت روی همه endpoints؛ حسابرسی exposure توکن	نشست access token از طریق endpoint اطلاعات سرور	CVE-2023-28432 (MinIO)

۳.۲ دسته‌بندی حملات رایج

حمله CSRF روی جریان‌های OAuth

رایج‌ترین آسیب‌پذیری OAuth. بدون پارامتر (CSRF token) state، مهاجم می‌تواند:

- جریان OAuth را شروع کند و authorization code بگیرد
- آن را تکمیل نکند (کد را تبادل نکند)
- لینک جعلی حاوی کد خود را برای قربانی ارسال کند
- اگر قربانی به کلاینت وارد شده باشد، session او با حساب مهاجم پیوند می‌خورد

حملات Open Redirect

اگر اعتبارسنجی redirect_uri خیلی سهل گیرانه باشد، مهاجم می تواند authorization code را به سرور خود هدایت کند. مثال:

- آسیب پذیر (prefix-only): `https://app.com/callback.evil.com` از بررسی prefix رد می شود
- آسیب پذیر (wildcard): `https://evil.app.com.attacker.com/callback`
- امن: تطابق دقیق URI الزامی است

نشت توکن از طریق Implicit Flow

Implicit flow توکن را مستقیم در URL fragment قرار می دهد

- توکن در تاریخچه مرورگر ذخیره می شود
- توکن در HTTP Referer header هنگام نوبری به صفحات خارجی نشت می کند
- توکن برای JavaScript صفحه قابل دسترسی است (خطر XSS)
- راه حل: به جای آن از Authorization Code + PKCE استفاده کنید

فیشینگ رضایت OAuth (حملات دولت-ملت)

بازیگران تهدید ملی (مانند APT29/Midnight Blizzard) جریان های رضایت OAuth را به سلاح تبدیل کرده اند:

- مهاجم یک اپ چند-tenant با ظاهر قانونی ثبت می کند
- اپ درخواست مجوزهای بالا می دهد: Files.ReadWrite.All، Mail.ReadWrite.All
- ایمیل فیشینگ قربانی را به صفحه رضایت واقعی Microsoft هدایت می کند
- قربانی «Accept» می زند – اپ مهاجم توکن دسترسی دائمی دریافت می کند
- حتی اگر قربانی رمز عبور را تغییر دهد، OAuth grant باقی می ماند
- مهاجم می تواند ایمیل، فایل ها و مخاطبین را بی صدا استخراج کند

بخش ۴: معماری و اجزای OAuth

۴.۱ موجودیت های اصلی OAuth 2.0

مثال	نقش	موجودیت
کاربر نهایی	کاربری که داده را مالک است و می تواند دسترسی اعطا کند	Resource Owner (مالک منبع)
Google Drive API, GitHub API	سرور API که منابع محافظت شده را میزبانی می کند؛ توکن ها را اعتبارسنجی می کند	Resource Server (سرور منبع)
اپ موبایل، وب اپ، CLI	برنامه ای که از طرف مالک منبع درخواست دسترسی می کند	Client (کلاینت)
Google Auth, Keycloak, Auth0, Okta	مالک منبع را احراز هویت می کند؛ توکن ها صادر می کند؛ رضایت مدیریت می کند	Authorization Server (سرور مجوز)

۴.۲ ساختار JWT (JSON Web Token)

توکن‌های دسترسی و ID در سیستم‌های مدرن OAuth معمولاً JWT هستند:

فیلد	نام	توضیح
iss	Issuer (صادرکننده)	URL سرور صادرکننده توکن
sub	Subject (موضوع)	شناسه منحصر به فرد کاربر
aud	Audience (مخاطب)	گیرنده مجاز توکن (client_id یا API)
exp	Expiration (انقضا)	زمان انقضای توکن (Unix timestamp)
iat	Issued At (صدور)	زمان صدور توکن
jti	JWT ID	شناسه منحصر به فرد توکن (برای ابطال)
scope	Scope (حوزه دسترسی)	مجوزهای اعطاشده

بخش ۵: بهترین شیوه‌های امنیتی

۵.۱ چک‌لیست امنیتی سمت سرور

کنترل امنیتی	پیاده‌سازی	چه چیزی جلوگیری می‌کند
تطابق دقیق Redirect URI	URI کامل شامل scheme، host، path را مقایسه کنید (بدون wildcard)	حملات open redirect، دزدی auth code
اجباری PKCE	code_challenge برای همه جریان‌های Authorization Code الزامی باشد	تزریق CSRF، auth code
پارامتر State	رشته تصادفی رمزنگاری شده (≥ ۱۲۸ بیت)، متصل به session	حملات CSRF روی جریان OAuth
Nonce در OIDC	مقدار تصادفی در درخواست مجوز؛ در claim id_token اعتبارسنجی شود	بازپخش ID token
طول عمر کوتاه توکن	Access token: ۵-۶۰ دقیقه؛ ID token: همان access token	پنجره آسیب از دزدی توکن
Refresh Token Rotation	Refresh token جدید در هر استفاده صادر شود؛ قدیمی ابطال شود؛ استفاده مجدد شناسایی شود	دزدی refresh token و دسترسی بلندمدت
Token Binding/DPoP	mTLS certificate binding یا DPoP (RFC 9449)	دزدی و بازپخش bearer token
اعتبارسنجی Scope	سرور تأیید کند scope‌های اعطاشده ≥ scope‌های درخواست‌شده	افزایش امتیاز از طریق دستکاری scope

۵.۲ چک‌لیست امنیتی سمت کلاینت

- هرگز access token را در localStorage ذخیره نکنید (XSS accessible) – از حافظه یا HttpOnly cookie استفاده کنید
- هرگز access token را در URL parameter یا fragment قرار ندهید
- پارامتر state را در هر OAuth callback اعتبارسنجی کنید
- همه claims JWT را اعتبارسنجی کنید: iss، aud، exp، iat، nonce قبل از اعتماد به توکن
- از کتابخانه‌های OAuth معتبر استفاده کنید؛ JWT parsing را دستی پیاده‌سازی نکنید
- همه توکن‌ها را در logout پاک کنید؛ refresh token را از طریق revocation endpoint باطل کنید
- برای اپ‌های native: از system browser استفاده کنید (/ ASWebAuthenticationSession
Chrome Custom Tabs)، نه WebView
- فقط scope‌های مورد نیاز را درخواست کنید

۵.۳ نگهداری امن توکن بر حسب پلتفرم

پلتفرم	Access Token	Refresh Token	ضدالگو (اجتناب کنید)
Web (SPA)	حافظه JavaScript	HttpOnly, Secure, SameSite=Strict cookie	localStorage، sessionStorage، URL fragment
Web (Server-Side)	حافظه سرور / session کوتاه‌مدت	Session store رمزگذاری‌شده سمت سرور	نگهداری client-side refresh token
موبایل (iOS)	Keychain	Keychain با محافظت biometric	UserDefaults، فایل‌های plist
موبایل (Android)	Android Keystore + EncryptedSharedPreferences	Keystore با strong box backing	SharedPreferences رمزنگاری‌نشده
Desktop/CLI	حافظه / OS credential store	OS credential manager	فایل‌های config، متنی، environment variables

بخش ۶: مفاهیم پیشرفته

۶.۱ OAuth به‌عنوان پروتکل تفویض، نه مجوزدهی

یک تمایز مفهومی مهم که اغلب نادیده گرفته می‌شود: OAuth اساساً یک پروتکل تفویض است، نه مجوزدهی. تشبیه: مدیری کاغذی امضا می‌کند که به کارمند اجازه برداشت از حساب بانکی شرکت را می‌دهد – این تفویض است. اما وقتی کارمند به بانک می‌رود، بانک ممکن است رد کند چون سیاست‌های داخلی خود را دارد – این مجوزدهی است.

- توکن OAuth نشان‌دهنده تفویض دسترسی است، نه تضمین دسترسی
- Resource Server (API) آخرین کلام را دارد

۶.۲ توکن‌های By-Value در مقابل By-Reference

معایب	مزایا	نحوه کار	نوع
API باید برای هر درخواست با Auth Server تماس بگیرد (introspection) – تأخیر بیشتر	حریم خصوصی – بدون PII در شبکه؛ قابل ابطال فوری	رشته تصادفی (مثل UUID) که به یک ورودی در DB اشاره دارد – محتوا قابل خواندن نیست	By-Reference (Opaque Token)
Claims برای هر کسی که توکن را دارد قابل خواندن است؛ ابطال فوری پیچیده است	API می‌تواند محلی اعتبارسنجی کند – تأخیر کمتر؛ برای شبکه داخلی مناسب	JWT امضا شده با تمام claims داخل آن – هر کسی می‌تواند payload را بخواند	By-Value (JWT)

توصیه: از opaque token در کانال‌های خارجی (اینترنت) و از JWT در کانال‌های داخلی پشت API Gateway استفاده کنید.

۶.۳ الگوی Phantom Token Flow

این الگو تنش بین حریم خصوصی و کارایی را حل می‌کند:

- کلاینت خارجی یک opaque token به Gateway ارسال می‌کند
- Gateway با introspection endpoint سرور مجوز تماس می‌گیرد
- سرور مجوز با یک JWT (نه JSON) پاسخ می‌دهد
- Gateway آن JWT را به API backend داخلی ارسال می‌کند
- API backend JWT را محلی اعتبارسنجی می‌کند – بدون تماس بیشتر با Auth Server

۶.۴ آسیب‌پذیری alg:none در JWT

JWT حاوی فیلد alg در header است که الگوریتم امضا را مشخص می‌کند. آسیب‌پذیری حیاتی:

- بعضی کتابخانه‌های JWT، header را بدون محدودیت می‌پذیرند
- مهاجم alg را روی «none» تنظیم می‌کند – کتابخانه‌های آسیب‌پذیر از اعتبارسنجی امضا صرف‌نظر می‌کنند
- مهاجم می‌تواند هر توکنی با هر claim جعل کند

توصیه	سطح امنیتی	الگوریتم
هرگز مجاز ندانید	آسیب‌پذیری حیاتی	none
از تولید اجتناب کنید	ضعیف در سیستم‌های توزیع‌شده	HS256 (مقارن)
استاندارد توصیه‌شده برای تولید	قوی – کلید خصوصی امضا، عمومی اعتبارسنجی	RS256 (نامقارن RSA)
ترجیح داده می‌شود برای سیستم‌های حساس به کارایی	قوی – کلیدهای کوچکتر از RSA، امنیت یکسان	ES256 (ECDSA) (نامقارن)
برای محیط‌های با بالاترین امنیت	قوی‌ترین نوع RSA	PS256 (RSA-PSS)

۶.۵ کلاینت نباید Access Token را decode کند

یک بهترین شیوه که اغلب نقض می‌شود: برنامه کلاینت نباید هرگز محتوای access token را decode یا بررسی کند، حتی اگر JWT باشد.

- access token یک قرارداد بین Authorization Server و Resource Server است – کلاینت طرف این قرارداد نیست
- اگر AS و API ساختار توکن را تغییر دهند، نیازی به اطلاع کلاینت ندارند
- اپ‌های موبایل که ساختار توکن را decode می‌کنند ممکن است برای تمام کاربران نسخه‌های قدیمی خراب شوند
- برای اطلاعات کاربر از ID token (OIDC) یا UserInfo endpoint استفاده کنید – این‌ها قرارداد پایداری برای کلاینت‌ها هستند

بخش ۷: راه‌حل‌های OAuth

۷.۱ سرورهای OAuth/OIDC متن‌باز

بهترین کاربرد	ویژگی‌های کلیدی	راه‌حل
سازمانی on-premises؛ سناریوهای پیچیده هویت	کامل‌ترین؛ RBAC؛ رابط ادمین؛ یکپارچه‌سازی LDAP/AD	Keycloak (Java)
API‌های پرتراфик؛ تیم‌هایی با UI سفارشی	Headless native؛ بسیار مقیاس‌پذیر؛ متن‌باز؛ cloud-	Hydra (Ory - Go)
Kubernetes؛ فدراسیون منابع هویت	OIDC provider upstream؛ پروکسی برای IdP‌های	Dex (Go)
تیم‌های self-hosted که UX مدرن می‌خواهند	رابط مدرن؛ موتور جریان؛ RBAC؛ FAY	Authentik (Python/Go)

۷.۲ راه‌حل‌های ابری/تجاری

ویژگی‌های برجسته	راه‌حل
یکپارچه‌سازی آسان؛ پایپ‌لاین rules/actions؛ تشخیص ناهنجاری؛ اتصال‌های اجتماعی گسترده	Auth0 (Okta)
دایرکتوری سازمانی؛ Conditional Access؛ managed identities؛ Privileged Identity Management	Microsoft Entra ID
Workforce Identity Federation؛ Identity-Aware Proxy؛ Firebase Auth	Google Identity Platform
machine-to-machine (M2M)؛ Lambda triggers؛ federated identities؛ User pools	AWS Cognito

این سند ترجمه و خلاصه فارسی مطالب کلیدی تحقیق OAuth می‌باشد / برای مطالعه کامل به مقاله اصلی مراجعه کنید

https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6454142